TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN

TÀI LIỆU THỰC HÀNH

Học máy ứng dụng

Bài 9: Mạng nơ-ron nhiều lớp (MLP)

Biên soạn ThS. Nguyễn Thái Anh

Năm học 2026 – 2027

Mục lục

1 Mục tiêu bài thực hành **2**
1.1 Chuẩn bị . . . 2

2 Tự tính lan truyền tiến **2**
2.1 Một nơ-ron . . . 3
2.2 Một mạng giải được XOR . . . 3
2.3 Đọc kết quả . . . 4

3 Một nơ-ron và một lớp ẩn trên XOR **5**
3.1 Đọc kết quả . . . 6

4 Độ lớn mạng và điều chuẩn **6**
4.1 Đọc bảng thứ nhất . . . 8
4.2 Chọn alpha bằng kiểm định chéo . . . 8

5 Mạng nơ-ron khi quên chuẩn hoá **9**
5.1 Đọc kết quả . . . 10

6 Nhận dạng chữ số viết tay **10**
6.1 Số tham số . . . 12
6.2 Đường mất mát . . . 12
6.3 Kết quả . . . 12

7 Bài tập **13**

8 Phụ lục A, bảng tra nhanh **15**

9 Phụ lục B, lỗi thường gặp **16**

1. Mục tiêu bài thực hành

Ở bài 2, hồi quy logistic tính một tổng có trọng số rồi đưa qua hàm sigmoid. Mỗi khối như vậy gọi là một **nơ-ron**. Buổi này ghép nhiều nơ-ron thành nhiều lớp để được một mạng nơ-ron, mô hình vẽ được ranh giới cong tuỳ ý và là nền móng của học sâu.

Sau khi hoàn thành bài này, sinh viên có khả năng:

1. Tự tính lan truyền tiến qua một nơ-ron và qua một mạng nhỏ bằng numpy.
2. Thấy vì sao một nơ-ron không giải được XOR, còn một lớp ẩn thì được.
3. Nhận ra mạng học vẹt và chọn độ nặng điều chuẩn `alpha` bằng kiểm định chéo.
4. Kiểm chứng rằng mạng nơ-ron bắt buộc chuẩn hoá.
5. Dùng `MLPClassifier` nhận dạng chữ số viết tay, đọc số tham số và đường mất mát.

Bốn bộ dữ liệu

Tệp `data/xor.csv` gồm 400 điểm, lớp 1 nằm ở góc trên phải và góc dưới trái, là số liệu mô phỏng. Dùng lại `data/hai_vang_trang.csv`, `data/sinh_vien_mo_rong.csv` và bộ chữ số viết tay `load_digits` của các bài trước.

1.1. Chuẩn bị

Vẫn môi trường của bài 1, không cài thêm gì.

```
cd C:\MayHoc
hocmay\Scripts\activate
cd bai09_mlp
```

Bảng 1: *Năm tệp mã của buổi thực hành, chạy theo đúng thứ tự này.*

Mục	Tệp	Nội dung
2	`m1_lan_truyen_tien.py`	Tự tính lan truyền tiến bằng numpy
3	`m2_xor.py`	Một nơ-ron và một lớp ẩn trên XOR
4	`m3_do_lon_mang.py`	Độ lớn mạng và điều chuẩn
5	`m4_chuan_hoa.py`	Mạng nơ-ron khi quên chuẩn hoá
6	`m5_chu_so.py`	Nhận dạng chữ số viết tay

Cảnh báo khi chạy

Khi mạng chưa hội tụ hết số vòng cho phép, scikit-learn in ra `ConvergenceWarning`. Đây là lời nhắc, không phải lỗi, chương trình vẫn chạy tiếp. Các tệp mã của bài đã đặt `max_iter` đủ lớn để hạn chế cảnh báo này.

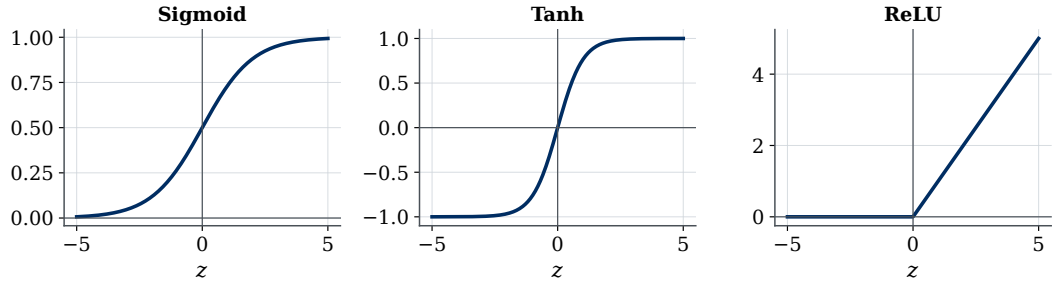
2. Tự tính lan truyền tiến

2.1. Một nơ-ron

Nơ-ron tính hai bước

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b \qquad a = g(z)$$

w là các trọng số, b là hệ số chặn, g là hàm kích hoạt. Nếu g là sigmoid thì nơ-ron chính là hồi quy logistic của bài 2.



Hình 1: *Ba hàm kích hoạt hay dùng. ReLU là mặc định cho lớp ẩn trong scikit-learn.*

Hàm ReLU là đơn giản nhất: $\mathrm{ReLU}(z) = \max(0, z)$, số âm thành 0, số dương giữ nguyên.

2.2. Một mạng giải được XOR

Mạng có 2 đầu vào, 2 nơ-ron ẩn dùng ReLU và 1 nơ-ron ra không dùng hàm kích hoạt:

$$h_1 = \max(0,\, x_1 + x_2) \qquad h_2 = \max(0,\, x_1 + x_2 - 1) \qquad y = h_1 - 2h_2$$

Các trọng số ở đây do ta tự đặt để thấy một mạng nhỏ có thể tính đúng phép XOR.

```python
# -*- coding: utf-8 -*-
"""Bước 1: tự tính lan truyền tiến bằng numpy.

Chú thích viết tiếng Việt có dấu, phần in ra màn hình viết không dấu.
"""

import numpy as np


def sigmoid(z):
    return 1 / (1 + np.exp(-z))


def relu(z):
    return np.maximum(0, z)


# Phần 1: một nơ-ron với hàm sigmoid, chính là hồi quy logistic
w = np.array([0.8, -0.5, 0.3])
b = -0.2
x = np.array([2.0, 1.0, 3.0])
z = w @ x + b
```

```python
print("Mot no-ron:")
print(f"  z = {w[0]}*{x[0]} + ({w[1]})*{x[1]} + {w[2]}*{x[2]}"
      f" + ({b}) = {z:.2f}")
print(f"  a = sigmoid(z) = {sigmoid(z):.4f}")
print()

# Phần 2: mạng 2 đầu vào, 2 nơ-ron ẩn ReLU, 1 nơ-ron ra
# Trọng số tự đặt để mạng tính đúng phép XOR
W1 = np.array([[1.0, 1.0],       # cột 1: trọng số vào nơ-ron ẩn h1
               [1.0, 1.0]])      # cột 2: trọng số vào nơ-ron ẩn h2
b1 = np.array([0.0, -1.0])
W2 = np.array([1.0, -2.0])
b2 = 0.0

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
print("Mang giai XOR:")
print("  x1  x2   h1  h2    y")
for x in X:
    h = relu(x @ W1 + b1)          # lớp ẩn
    y = h @ W2 + b2                # lớp ra
    print(f"  {x[0]:.0f}   {x[1]:.0f}    {h[0]:.0f}   {h[1]:.0f}"
          f"    {y:.0f}")
print()

# Cả bốn đầu vào cùng lúc bằng phép nhân ma trận
H = relu(X @ W1 + b1)
Y = H @ W2 + b2
print("Tinh ca bang mot lan:", Y)
```

```
python code/m1_lan_truyen_tien.py
```

```
Mot no-ron:
  z = 0.8*2.0 + (-0.5)*1.0 + 0.3*3.0 + (-0.2) = 1.80
  a = sigmoid(z) = 0.8581

Mang giai XOR:
  x1  x2   h1  h2    y
  0   0    0   0    0
  0   1    1   0    1
  1   0    1   0    1
  1   1    2   1    0

Tinh ca bang mot lan: [0. 1. 1. 0.]
```

2.3. Đọc kết quả

Với nơ-ron ở phần 1: $z = 1.6 - 0.5 + 0.9 - 0.2 = 1.80$, và $\text{sigmoid}(1.80) \approx 0.8581$.

Với mạng ở phần 2, cột y đúng bằng XOR: 0, 1, 1, 0. Ở đầu vào $(1, 1)$, nơ-ron h_2 mới bật lên và trừ bớt đi, kéo y về 0. Chính hàm ReLU bẻ gãy đường thẳng này, nhờ đó mạng làm được

điều một đường thẳng không làm được.

Tính cả bảng bằng phép nhân ma trận

Phép `X @ W1` nhân ma trận dữ liệu 4 hàng 2 cột với ma trận trọng số 2 hàng 2 cột, được 4 hàng 2 cột, mỗi hàng là z của hai nơ-ron ẩn. Thư viện học máy tính mạng nơ-ron đúng theo cách này, cho cả lô dữ liệu cùng lúc.

3. Một nơ-ron và một lớp ẩn trên XOR

Mục 2 tự đặt trọng số. Giờ để `MLPClassifier` tự học trọng số từ 300 điểm của tập học, và so với hồi quy logistic.

```python
# -*- coding: utf-8 -*-
"""Bước 2: một nơ-ron không giải được XOR, một lớp ẩn thì được."""

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

df = pd.read_csv("data/xor.csv")
X = df[["x1", "x2"]]
y = df["nhan"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

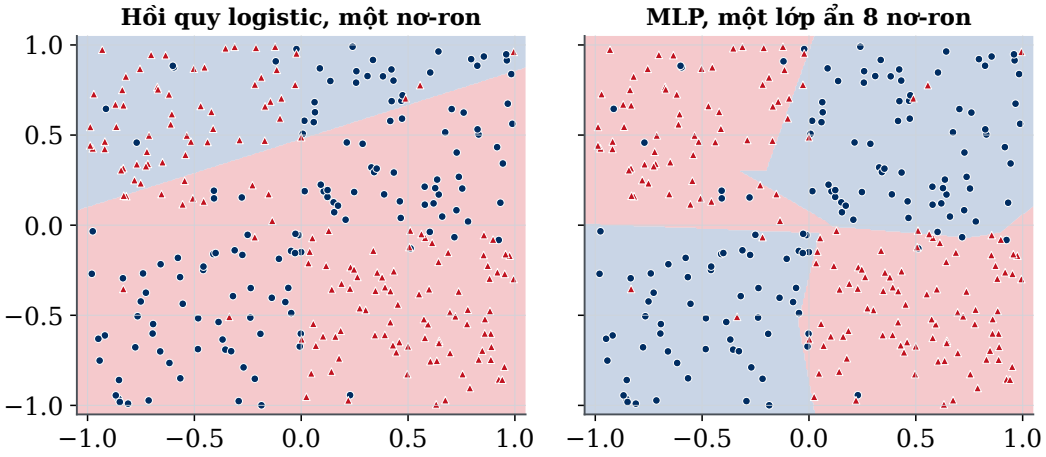
lr = LogisticRegression().fit(X_tr, y_tr)
print(f"Hoi quy logistic: {accuracy_score(y_te, lr.predict(X_te)):.4f}")
print()

# Dữ liệu nhỏ nên dùng solver="lbfgs", học nhanh và ổn định hơn
print("So no-ron an   so tham so   tap hoc   tap kiem tra")
for so in [1, 2, 4, 8]:
    mlp = MLPClassifier(hidden_layer_sizes=(so,), solver="lbfgs",
                        max_iter=5000, random_state=0)
    mlp.fit(X_tr, y_tr)
    # coefs_ là danh sách ma trận trọng số, intercepts_ là hệ số chặn
    so_ts = sum(w.size for w in mlp.coefs_)
    so_ts += sum(b.size for b in mlp.intercepts_)
    a_hoc = accuracy_score(y_tr, mlp.predict(X_tr))
    a_kt = accuracy_score(y_te, mlp.predict(X_te))
    print(f"     {so:2d}          {so_ts:3d}       {a_hoc:.4f}"
          f"    {a_kt:.4f}")
```

```
python code/m2_xor.py
```

```
Hoi quy logistic: 0.4500

So no-ron an   so tham so   tap hoc   tap kiem tra
      1             5       0.6533    0.6900
      2             9       0.6533    0.6900
      4            17       0.9033    0.9300
      8            33       0.9167    0.9100
```



Hình 2: *Ranh giới của hồi quy logistic và của MLP một lớp ẩn 8 nơ-ron trên bộ XOR.*

3.1. Đọc kết quả

Hồi quy logistic chỉ được 0.4500 trên tập kiểm tra, còn kém hơn tung đồng xu. Một đường thẳng cắt ngang thế nào cũng để hai góc cùng lớp nằm về hai phía.

Với lớp ẩn:

- 1 hoặc 2 nơ-ron ẩn: chỉ được 0.6900, mạng quá nhỏ.
- 4 nơ-ron ẩn: 0.9300.
- 8 nơ-ron ẩn: 0.9100, ranh giới chia mặt phẳng thành bốn góc như Hình 2.

Đếm số tham số

Với 8 nơ-ron ẩn: lớp ẩn có 2×8 trọng số và 8 hệ số chặn, lớp ra có 8 trọng số và 1 hệ số chặn, tổng cộng $16 + 8 + 8 + 1 = 33$. Thuộc tính `coefs_` là danh sách các ma trận trọng số, `intercepts_` là danh sách các hệ số chặn.

Vì sao dùng solver lbfgs

Mặc định `MLPClassifier` dùng `solver="adam"`, hợp với dữ liệu lớn. Với vài trăm mẫu, `solver="lbfgs"` thường học nhanh và ổn định hơn, đúng như tài liệu của scikit-learn gợi ý.

4. Độ lớn mạng và điều chuẩn

Mạng càng nhiều nơ-ron càng vẽ được ranh giới phức tạp, và càng dễ học vẹt. Để kìm lại, `MLPClassifier` có tham số `alpha`.

Điều chuẩn L2

Mất mát khi học được cộng thêm một khoản phạt:

$$L_{\text{mới}} = L + \alpha \times (\text{tổng bình phương mọi trọng số})$$

α càng lớn, mạng càng bị ép giữ trọng số nhỏ, ranh giới càng mượt.

```python
# -*- coding: utf-8 -*-
"""Bước 3: mạng to thì học vẹt, điều chuẩn bằng alpha thì đỡ."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/hai_vang_trang.csv")
X = df[["x1", "x2"]]
y = df["nhan"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)


def mang(lop_an, alpha):
    return make_pipeline(
        StandardScaler(),
        MLPClassifier(hidden_layer_sizes=lop_an, alpha=alpha,
                      solver="lbfgs", max_iter=20000, random_state=0),
    )


print("Ba mang              tap hoc   tap kiem tra")
for ten, lop_an, alpha in [("(2,), alpha 1e-4", (2,), 1e-4),
                           ("(100, 100), 1e-5", (100, 100), 1e-5),
                           ("(100, 100), 1.0", (100, 100), 1.0)]:
    m = mang(lop_an, alpha).fit(X_tr, y_tr)
    print(f"  {ten:18s} {accuracy_score(y_tr, m.predict(X_tr)):.4f}"
          f"    {accuracy_score(y_te, m.predict(X_te)):.4f}")
print()

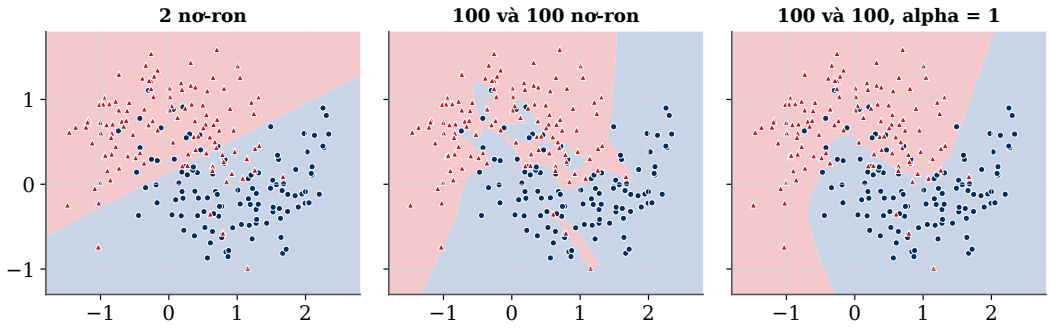
# Chọn alpha bằng kiểm định chéo, chỉ trên tập học
print("alpha      kiem dinh cheo")
diem = {}
for alpha in [1e-5, 1e-3, 0.1, 1.0, 3.0, 10.0]:
    diem[alpha] = cross_val_score(mang((100, 100), alpha),
                                  X_tr, y_tr, cv=5).mean()
    print(f"  {alpha:<8}  {diem[alpha]:.4f}")
```

```
a_tot = max(diem, key=diem.get)
m = mang((100, 100), a_tot).fit(X_tr, y_tr)
print(f"alpha tot nhat = {a_tot}, tap kiem tra "
      f"{accuracy_score(y_te, m.predict(X_te)):.4f}")
```

```
python code/m3_do_lon_mang.py
```

```
Ba mang              tap hoc   tap kiem tra
  (2,), alpha 1e-4   0.8444    0.8533
  (100, 100), 1e-5   1.0000    0.7333
  (100, 100), 1.0    0.8800    0.9067

alpha      kiem dinh cheo
  1e-05     0.7778
  0.001     0.7733
  0.1       0.7644
  1.0       0.8578
  3.0       0.8400
  10.0      0.8356
alpha tot nhat = 1.0, tap kiem tra 0.9067
```



Hình 3: *Ranh giới của ba mạng trên bộ hai vầng trăng.*

4.1. Đọc bảng thứ nhất

- Mạng 2 nơ-ron: 0.8444 và 0.8533, ranh giới gần như thẳng, mạng **cẩu thả**.
- Mạng hai lớp 100 nơ-ron với `alpha` gần như bằng 0: tập học 1.0000 nhưng tập kiểm tra chỉ 0.7333. Ranh giới trong Hình 3 uốn lượn quanh từng điểm, mạng **học vẹt**.
- Cùng mạng đó với `alpha=1.0`: tập học 0.8800, tập kiểm tra 0.9067. Ranh giới mượt mà theo đúng hình hai vầng trăng.

4.2. Chọn alpha bằng kiểm định chéo

Bảng thứ hai thử sáu giá trị `alpha` bằng kiểm định chéo 5 phần, chỉ trên tập học. Điểm cao nhất là 0.8578 ở `alpha=1.0`. Chọn xong mới chấm tập kiểm tra một lần, được 0.9067.

Cùng một ý tưởng đã gặp nhiều lần

k của KNN, độ sâu của cây, C và gamma của SVM, giờ là `alpha` và độ lớn của mạng. Tất cả đều là nút vặn giữa cẩu thả và học vẹt, và đều chọn bằng kiểm định chéo.

5. Mạng nơ-ron khi quên chuẩn hoá

Bộ sinh viên mở rộng có cột chi tiêu tính bằng nghìn đồng, lớn gấp hàng trăm lần hai cột giờ ôn và điểm giữa kỳ. Bài 4 và bài 6 đã thấy KNN và SVM gặp khó với bộ này.

```python
# -*- coding: utf-8 -*-
"""Bước 4: mạng nơ-ron khi quên chuẩn hoá."""

import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/sinh_vien_mo_rong.csv")
X = df[["gio_on", "diem_giua_ky", "chi_tieu"]]
y = df["qua_mon"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

tho = MLPClassifier(hidden_layer_sizes=(20,), max_iter=3000,
                    random_state=0).fit(X_tr, y_tr)
chuan = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(20,), max_iter=3000,
                  random_state=0),
).fit(X_tr, y_tr)

print(f"Doan bua ca lop qua : {y_te.mean():.4f}")
print(f"Khong chuan hoa     : "
      f"{accuracy_score(y_te, tho.predict(X_te)):.4f}")
print(f"Co chuan hoa        : "
      f"{accuracy_score(y_te, chuan.predict(X_te)):.4f}")
print()

# Mạng không chuẩn hoá đoán ra sao
du = tho.predict(X_te)
print("Khong chuan hoa, so ban doan qua:", du.sum(), "tren", len(du))
```

```
python code/m4_chuan_hoa.py
```

```
Doan bua ca lop qua : 0.6000
Khong chuan hoa     : 0.4000
Co chuan hoa        : 0.9000

Khong chuan hoa, so ban doan qua: 0 tren 30
```

5.1. Đọc kết quả

Không chuẩn hoá, mạng chỉ đạt 0.4000, còn tệ hơn đoán bừa 0.6000. Dòng cuối cho thấy lý do: mạng đoán **mọi** bạn đều rớt, 0 trên 30 bạn được đoán qua, nên accuracy đúng bằng tỷ lệ bạn rớt. Có chuẩn hoá, mạng đạt 0.9000.

Mạng nơ-ron nhạy với thang đo hơn cả KNN

Cột có giá trị hàng nghìn làm z của các nơ-ron rất lớn ngay từ những bước đầu. Độ dốc bị cột đó chi phối, và giảm độ dốc không tìm được hướng đi đúng. Luôn dùng `make_pipeline(StandardScaler(), MLPClassifier())`.

6. Nhận dạng chữ số viết tay

Mục cuối dùng MLP cho bộ chữ số viết tay của bài 6: 64 đầu vào là 64 ô độ sáng, 10 nơ-ron ra ứng với 10 chữ số. Với dữ liệu này, dùng `solver="adam"` mặc định.

```python
# -*- coding: utf-8 -*-
"""Bước 5: nhận dạng chữ số viết tay bằng MLP."""

import numpy as np
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=17, stratify=y
)

mang = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(100,), max_iter=1000,
                  random_state=0),
).fit(X_tr, y_tr)
mlp = mang[-1]
```

```
26
27 print("Kich thuoc cac ma tran trong so:",
28       [w.shape for w in mlp.coefs_])
29 so_ts = sum(w.size for w in mlp.coefs_)
30 so_ts += sum(b.size for b in mlp.intercepts_)
31 print("Tong so tham so:", so_ts)
32 print("So vong hoc:", mlp.n_iter_)
33 mm = mlp.loss_curve_
34 print(f"Mat mat vong 1: {mm[0]:.4f}, vong 10: {mm[9]:.4f},"
35       f" vong cuoi: {mm[-1]:.4f}")
36 print()
37
38 du = mang.predict(X_te)
39 print(f"Accuracy tren tap kiem tra: {accuracy_score(y_te, du):.4f}")
40 sai = np.flatnonzero(du != y_te)
41 print("So anh doan sai:", len(sai), "tren", len(y_te))
42 for i in sai:
43     print(f"  anh {i:3d}: that la {y_te[i]}, doan la {du[i]}")
44 print()
45
46 # Dừng sớm: giữ riêng 10% tập học để theo dõi
47 dung_som = make_pipeline(
48     StandardScaler(),
49     MLPClassifier(hidden_layer_sizes=(100,), max_iter=1000,
50                   early_stopping=True, random_state=0),
51 ).fit(X_tr, y_tr)
52 print("Dung som sau", dung_som[-1].n_iter_, "vong, accuracy",
53       f"{accuracy_score(y_te, dung_som.predict(X_te)):.4f}")
54 print()
55
56 # So với các mô hình đã học, cùng tập học và tập kiểm tra
57 khac = {
58     "Logistic co chuan hoa": make_pipeline(
59         StandardScaler(), LogisticRegression(max_iter=2000)),
60     "KNN, k = 5": KNeighborsClassifier(),
61     "SVM kernel RBF": SVC(),
62 }
63 for ten, mh in khac.items():
64     acc = accuracy_score(y_te, mh.fit(X_tr, y_tr).predict(X_te))
65     print(f"{ten:22s} {acc:.4f}")
```

```
python code/m5_chu_so.py
```

```
Kich thuoc cac ma tran trong so: [(64, 100), (100, 10)]
Tong so tham so: 7510
So vong hoc: 144
Mat mat vong 1: 2.5206, vong 10: 0.4448, vong cuoi: 0.0052

Accuracy tren tap kiem tra: 0.9844
```

```
So anh doan sai: 7 tren 450
  anh   6: that la 9, doan la 3
  anh  61: that la 5, doan la 7
  anh  77: that la 2, doan la 1
  anh  85: that la 9, doan la 7
  anh 276: that la 8, doan la 2
  anh 428: that la 7, doan la 9
  anh 443: that la 4, doan la 7

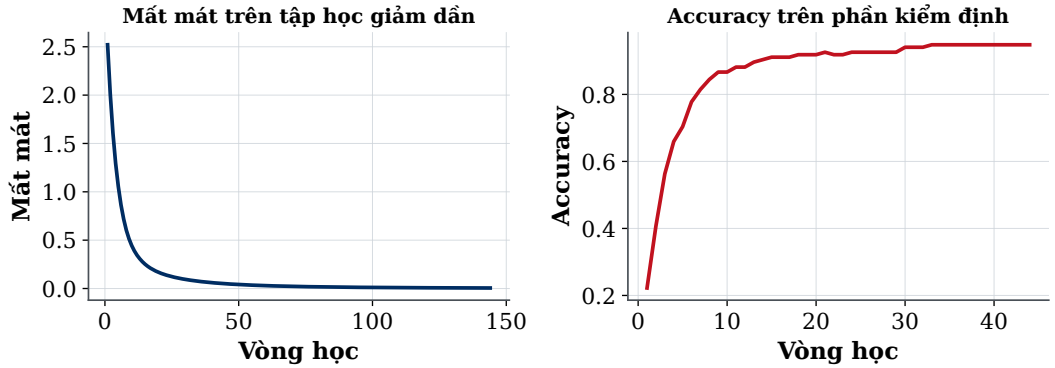
Dung som sau 44 vong, accuracy 0.9578

Logistic co chuan hoa  0.9711
KNN, k = 5             0.9867
SVM kernel RBF         0.9911
```

6.1. Số tham số

Hai ma trận trọng số có kích thước 64×100 và 100×10. Cộng thêm 100 và 10 hệ số chặn, mạng có $6400 + 100 + 1000 + 10 = 7510$ tham số phải học. Hồi quy logistic cho cùng bài toán chỉ có $64 \times 10 + 10 = 650$ tham số.

6.2. Đường mất mát



Hình 4: *Trái: mất mát trên tập học qua từng vòng. Phải: accuracy trên phần kiểm định khi bật dừng sớm.*

Mất mát giảm từ 2.5206 ở vòng 1 xuống 0.4448 ở vòng 10, rồi xuống 0.0052 ở vòng cuối. Mạng dừng sau 144 vòng vì mất mát không còn giảm đáng kể nữa.

6.3. Kết quả

Accuracy trên tập kiểm tra là 0.9844, sai 7 trên 450 ảnh. Hai ảnh số 6 và số 428 cũng chính là hai ảnh SVM đoán sai ở bài 6, một chữ 9 viết giống chữ 3 và một chữ 7 viết giống chữ 9.

Bảng 2: *Các mô hình trên cùng tập kiểm tra của bộ chữ số viết tay.*

Mô hình	Accuracy
Hồi quy logistic có chuẩn hoá	0.9711
MLP một lớp ẩn 100 nơ-ron	0.9844
KNN, $k = 5$	0.9867
SVM kernel RBF	0.9911

Dừng sớm không phải lúc nào cũng tốt hơn

Với `early_stopping=True`, mạng giữ riêng 10% tập học để theo dõi và dừng ở vòng 44. Accuracy chỉ còn 0.9578, vì mạng học trên ít ảnh hơn và dừng khá sớm. Dừng sớm hữu ích khi mạng có dấu hiệu học vẹt, còn ở đây mạng chưa học vẹt.

Mạng nơ-ron không phải lúc nào cũng thắng

Với 1797 ảnh nhỏ, SVM và KNN vẫn nhỉnh hơn MLP. Mạng nơ-ron thật sự vượt trội khi có rất nhiều dữ liệu, như hàng triệu ảnh lớn, và khi dùng những kiểu mạng đặc biệt của học sâu.

7. Bài tập

Sinh viên tạo thư mục `baitap09` nằm ngay trong thư mục bài thực hành. Vẫn chạy lệnh từ thư mục gốc, tức gõ `python baitap09/bai1.py`, và trong mã vẫn viết `data/...`. Bài nộp gồm tệp mã nguồn và ảnh chụp màn hình phần kết quả.

Bài tập 1: Tự dự đoán bằng trọng số đã học

Học lại mạng 8 nơ-ron ẩn trên bộ XOR như mục 3. Lấy `mlp.coefs_` và `mlp.intercepts_`, rồi tự tính lan truyền tiến bằng numpy cho 5 điểm đầu của tập kiểm tra: lớp ẩn dùng ReLU, lớp ra dùng sigmoid.
So kết quả với `mlp.predict_proba`, cột thứ hai. Hai kết quả phải khớp tới nhiều chữ số thập phân.

Bài tập 2: Một mạng XOR khác

Kiểm chứng bằng numpy rằng mạng sau cũng tính đúng XOR:

$$h_1 = \max(0,\, x_1 - x_2) \qquad h_2 = \max(0,\, x_2 - x_1) \qquad y = h_1 + h_2$$

Giải thích bằng lời vì sao mạng này đúng.

Bài tập 3: So các hàm kích hoạt

Trên bộ hai vầng trăng, dùng mạng hai lớp ẩn 100 nơ-ron với `alpha=1.0` như mục 4. Lần lượt đặt `activation` bằng `"logistic"`, `"tanh"` và `"relu"`. Ghi lại accuracy trên tập học và tập kiểm tra. Hàm nào cho kết quả tốt nhất ở đây?

Bài tập 4: Tốc độ học

Trên bộ chữ số, dùng mạng một lớp ẩn 100 nơ-ron, lần lượt đặt `learning_rate_init` bằng 0.0001, 0.001, 0.01 và 0.1. Với mỗi giá trị, ghi số vòng học `n_iter_`, mất mát cuối cùng và accuracy trên tập kiểm tra. Vẽ bốn đường `loss_curve_` lên cùng một hình và lưu thành `toc_do_hoc.png`. Tốc độ học quá lớn và quá nhỏ gây ra chuyện gì?

Bài tập 5: Dò lưới cho mạng nơ-ron

Dùng `GridSearchCV` trên bộ chữ số với `hidden_layer_sizes` trong `(10,)`, `(50,)`, `(100,)`, `(50, 50)` và `alpha` trong 0.0001, 0.01, 1. Mô hình là `make_pipeline(StandardScaler(), MLPClassifier(...))`, nên tên tham số trong lưới viết là `"mlpclassifier__alpha"`.
Cặp nào tốt nhất? Accuracy trên tập kiểm tra có vượt được SVM không?

Bài tập 6: Mạng nơ-ron cho hồi quy

Trên dữ liệu giá nhà của bài 1, dùng `MLPRegressor(hidden_layer_sizes=(20,), max_iter=5000, random_state=0)` có chuẩn hoá. So MAE trên tập kiểm tra với hồi quy tuyến tính. Dự đoán giá căn 250 m^2. Mạng nơ-ron có ngoại suy tốt như đường thẳng không?

Tiêu chí chấm

Mã chạy được không lỗi, kết quả in ra đúng, có nhận xét bằng lời ở những bài yêu cầu, tên biến đặt có nghĩa. Bài 1 và bài 2 chấm nặng phần giải thích, vì đó là chỗ kiểm tra sinh viên có hiểu mạng nơ-ron thật sự tính gì bên trong hay không.

8. Phụ lục A, bảng tra nhanh

Bảng 3: *Các hàm mới xuất hiện trong bài 9.*

Lệnh	Thư viện	Công dụng
`MLPClassifier`	scikit-learn	Mạng nơ-ron nhiều lớp cho bài toán phân loại.
`MLPRegressor`	scikit-learn	Mạng nơ-ron nhiều lớp cho bài toán hồi quy.
`hidden_layer_sizes`	scikit-learn	Số nơ-ron của từng lớp ẩn.
`activation`	scikit-learn	Hàm kích hoạt của lớp ẩn, mặc định `"relu"`.
`alpha`	scikit-learn	Độ nặng của điều chuẩn L2.
`solver`	scikit-learn	Cách tối ưu, `"adam"` hoặc `"lbfgs"`.
`early_stopping`	scikit-learn	Giữ riêng một phần tập học để dừng sớm.
`.coefs_`, `.intercepts_`	scikit-learn	Các ma trận trọng số và các hệ số chặn đã học.
`.loss_curve_`	scikit-learn	Mất mát qua từng vòng học, chỉ có với `"adam"`.
`.n_iter_`	scikit-learn	Số vòng học đã chạy.
`np.maximum`	numpy	Lấy giá trị lớn hơn, dùng để viết ReLU.
`@`	Python	Phép nhân ma trận.

Bảng 4: *Các con số kết quả chính của bài thực hành.*

Đại lượng	Giá trị
XOR, hồi quy logistic	0.4500
XOR, MLP 4 và 8 nơ-ron ẩn	0.9300 và 0.9100
Hai vầng trăng, mạng to không điều chuẩn	1.0000 và 0.7333
Hai vầng trăng, mạng to với alpha bằng 1	0.8800 và 0.9067
Sinh viên mở rộng, không và có chuẩn hoá	0.4000 và 0.9000
Chữ số, số tham số của mạng	7510
Chữ số, số vòng học	144
Chữ số, accuracy trên tập kiểm tra	0.9844, sai 7 trên 450

9. Phụ lục B, lỗi thường gặp

Bảng 5: *Các lỗi hay gặp riêng của bài mạng nơ-ron.*

Thông báo lỗi hoặc hiện tượng	Nguyên nhân và cách xử lý
`ConvergenceWarning: Stochastic Optimizer: Maximum iterations reached`	Mạng chưa hội tụ hết số vòng cho phép. Tăng `max_iter`, hoặc kiểm tra lại đã chuẩn hoá chưa.
Mạng đoán mọi mẫu vào cùng một lớp	Thường do quên chuẩn hoá. Dùng `StandardScaler` trong `make_pipeline`.
Mỗi lần chạy ra một kết quả	Trọng số ban đầu ngẫu nhiên. Đặt `random_state`.
Tập học gần 1.0, tập kiểm tra thấp	Mạng học vẹt. Tăng `alpha`, dùng mạng nhỏ hơn, hoặc bật `early_stopping`.
`AttributeError: 'MLPClassifier' object has no attribute 'loss_curve_'`	Đang dùng `solver="lbfgs"`. Đường mất mát chỉ có với `"adam"` và `"sgd"`.
Học rất chậm	Mạng quá lớn, hoặc tốc độ học quá nhỏ. Bắt đầu với một lớp ẩn vừa phải.

Một lời khuyên chung

Bắt đầu với một lớp ẩn 100 nơ-ron, có chuẩn hoá, để mặc định. So với hồi quy logistic và rừng ngẫu nhiên trước khi kết luận mạng nơ-ron là tốt nhất.